

Text as Data HW 2

Clarke Campbell

Text as Data: Homework 2

Clarke Campbell

October 2, 2026

Pre-Existing Tutorial Information

```
# In these code chunks, I am loading and setting up the data
# like it was done in the tutorial.
library(readr)
library(dplyr)
library(tidyr)
library(stringr)
library(tidytext)
library(ggplot2)
library(forcats)
library(tibble)
library(scales)
```

```
file_a <- "texts/A07594__Circle_of_Commerce.txt"
file_b <- "texts/B14801__Free_Trade.txt"
file_c <- "texts/A06785.txt"

# Reading the raw text files into R.
text_a <- read_file(file_a)
text_b <- read_file(file_b)
text_c <- read_file(file_c)

# Combining into a tibble for tidytext workflows.
```

```
texts <- tibble(
  doc_title = c("Text A", "Text B", "Text C"),
  text = c(text_a, text_b, text_c)
)
```

```
texts
```

```
# A tibble: 3 x 2
  doc_title text
  <chr>      <chr>
1 Text A    "THE CIRCLE OF COMMERCE. The Prooeme. HERODOTVS in his CLIQ, report~
2 Text B    "CAP. I. The Causes of the want of Money in England. IT hauing plea~
3 Text C    "STrong imagination, nourished by opinion, may cause most men to su~
```

```
# Here, I am creating a data set of stopwords (including custom ones).
```

```
data("stop_words")
```

```
custom_stopwords <- tibble(
  word = c(
    "vnto", "haue", "doo", "hath", "bee", "ye", "thee"
  )
)
```

```
all_stopwords <- bind_rows(stop_words, custom_stopwords) %>%
  distinct(word)
```

Section I

```
words <- texts |>
  unnest_tokens(word, text)
```

```
# Here, I am removing the stopwords from my broader 'words' tibble
# and normalizing the long s.
```

```
words_stopwords_removed <- words |>
  mutate(word = str_to_lower(word),
         word = str_replace_all(word, "f", "s")
  )|>
  anti_join(all_stopwords, by = "word")
```

```
words_stopwords_removed
```

```
# A tibble: 34,995 x 2
  doc_title word
  <chr>      <chr>
1 Text A    circle
2 Text A    commerce
3 Text A    prooeme
4 Text A    herodotvs
5 Text A    clio
6 Text A    reportes
7 Text A    croesvs
8 Text A    king
9 Text A    lydia
10 Text A   sonne
# i 34,985 more rows
```

```
# Here, I am loading the Bing dictionary and joining it with my
# stopword-free token data sets.
```

```
bing <- get_sentiments("bing")
```

```
words_bing_sentiment <- words_stopwords_removed |>
  inner_join(bing, by = "word", relationship = "many-to-many")
```

```
# Here, I am counting the positive and negative words,
# displaying them in a wider format, and showing net sentiment.
```

```
words_bing_sentiment_wide <- words_bing_sentiment |>
  group_by(doc_title, sentiment) |>
  count() |>
  ungroup() |>
  pivot_wider(names_from = sentiment, values_from = n) |>
  relocate(positive, .after = doc_title) |>
  mutate(net_sentiment = (positive - negative)) |>
  filter(doc_title != "Text C")
```

```
words_bing_sentiment_wide
```

```
# A tibble: 2 x 4
  doc_title positive negative net_sentiment
  <chr>         <int>    <int>         <int>
1 Text A         10      10             0
2 Text C         10      10             0
```

1 Text A	489	422	67
2 Text B	344	281	63

Section II

```
library(quanteda)
```

Warning: package 'quanteda' was built under R version 4.6.1

Package version: 4.5.0

Unicode version: 14.0

ICU version: 71.1

Parallel computing: disabled

See <https://quanteda.io> for tutorials and examples.

```
library(quanteda.textstats)

# Creating a quanteda corpus of the three documents;
# Treating long s as short s for parity with previous processing
threedoc_corpus <- corpus(
  gsub("f", "s", texts$text, fixed = TRUE)
)

# Removing punctuation, symbols, and all stopwords, and
# lowercasing tokens to match my tidytext preprocessing
# decisions - not removing numbers, because I didn't remove
# those earlier
threedoc_tokens <- tokens(threedoc_corpus,
  remove_punct = TRUE,
  remove_symbols = TRUE)

threedoc_tokens <- tokens_tolower(threedoc_tokens)

threedoc_tokens <- tokens_remove(threedoc_tokens, pattern = all_stopwords$word)

threedoc_tokens
```

Tokens consisting of 3 documents.

```
text1 :
  [1] "circle"      "commerce"  "prooeme"   "herodotvs" "clio"      "reportes"
  [7] "croesvs"     "king"      "lydia"     "sonne"     "borne"     "dumbe"
[ ... and 13,310 more ]
```

```
text2 :
  [1] "cap"      "money"    "england" "hauing"   "pleased" "god"      "giue"
  [8] "mee"      "birth"    "land"     "vnder"    "reigne"
[ ... and 7,885 more ]
```

```
text3 :
  [1] "strong"      "imagination" "nourished"  "opinion"    "suppose"
  [6] "edward"      "misselden"   "sallust"    "allenger"   "babylon"
 [11] "learning"    "seemeth"
[ ... and 13,581 more ]
```

```
# Creating a document feature matrix for the three tokenized documents
dfm_threedocs <- dfm(threedoc_tokens)

dfm_threedocs
```

Document-feature matrix of: 3 documents, 8,347 features (55.33% sparse) and 0 docvars.

```
      features
docs  circle commerce prooeme herodotvs clio reportes croesvs king lydia
text1    14        25      1          1  2          1          1  35    1
text2     1        22      0          0  0          0          0  53    0
text3    41        31      0          0  0          0          0  36    0
      features
docs  sonne
text1    5
text2    0
text3    0
[ reached max_nfeat ... 8,337 more features ]
```

```
# Checking the top words in the document feature matrix;
# they seem plausible
topfeatures(dfm_threedocs, 25)
```

```
trade    exchange commodities    money  merchants    moneys
```

602	478	293	290	282	267
kingdome	thereof	malynes	cloth	time	price
188	163	162	162	147	146
hee	king	mony	vpon	doth	countries
131	124	124	122	110	105
true	matter	4	english	merchant	england
104	103	97	94	92	91
low					
88					

```
# Calculating the document feature matrix TF-IDF scores
dfm_tfidf_threedocs <- dfm_tfidf(dfm_threedocs)

# Converting to a dataframe, pivoting to long, removing Text C
words_tfidf_twodocs <- convert(dfm_tfidf_threedocs, to = "data.frame") |>
  pivot_longer(cols = !doc_id, names_to = "word", values_to = "tf_idf") |>
  filter(doc_id != "text3")

# Joining the TF-IDF words with Bing sentiments

words_tfidf_sentiment <- inner_join(words_tfidf_twodocs, bing, by = "word", relationship = "many-to-many")
  arrange(desc(tf_idf))

words_tfidf_sentiment
```

```
# A tibble: 1,580 x 4
  doc_id word      tf_idf sentiment
  <chr>  <chr>      <dbl>  <chr>
1 text1  passion    4.29  positive
2 text1  divine     3.34  positive
3 text2  imposition  3.34  negative
4 text1  impertinent 2.86  negative
5 text1  affection   2.86  positive
6 text2  disorderly  2.86  negative
7 text1  imperfect   2.39  negative
8 text1  humane      1.91  positive
9 text1  meddle      1.91  negative
10 text1  kill        1.43  negative
# i 1,570 more rows
```

```
# Summing TF-IDF sentiments, pivoting wider
words_tfidf_sentiment_wide <- words_tfidf_sentiment |>
  group_by(doc_id, sentiment) |>
  reframe(tf_idf_totals = sum(tf_idf)) |>
  pivot_wider(names_from = sentiment, values_from = tf_idf_totals) |>
  relocate(positive, .after = doc_id) |>
  rename("positive_tf_idf_sum" = positive) |>
  rename("negative_tf_idf_sum" = negative) |>
  mutate(net_tf_idf_sentiment = (positive_tf_idf_sum - negative_tf_idf_sum))

words_tfidf_sentiment_wide
```

```
# A tibble: 2 x 4
  doc_id positive_tf_idf_sum negative_tf_idf_sum net_tf_idf_sentiment
  <chr>           <dbl>           <dbl>           <dbl>
1 text1             63.5             85.9            -22.4
2 text2             32.7             42.8            -10.1
```

Section III

```
# Setting 'doc_title' equal and joining summary tables
words_tfidf_and_count <- words_tfidf_sentiment_wide |>
  mutate(doc_title = case_when(doc_id == "text1" ~ "Text A", doc_id == "text2" ~ "Text B" ))
  select(!doc_id) |>
  relocate(doc_title, .before = positive_tf_idf_sum) |>
  inner_join(words_bing_sentiment_wide, by = join_by(doc_title))

words_tfidf_and_count
```

```
# A tibble: 2 x 7
  doc_title positive_tf_idf_sum negative_tf_idf_sum net_tf_idf_sentiment
  <chr>           <dbl>           <dbl>           <dbl>
1 Text A             63.5             85.9            -22.4
2 Text B             32.7             42.8            -10.1
# i 3 more variables: positive <int>, negative <int>, net_sentiment <int>
```

```
# Exporting the joined summary table as a CSV

write_csv(words_tfidf_and_count, "words_tfidf_and_count.csv")
```

Section IV

1. What changed between the two methods?

Each of the methods suggests something different: by raw count, in both documents, there are more positive sentiment words than negative sentiment words, but when weighted for TF-IDF significance, there is more negative sentiment than positive sentiment across both texts.

2. Why did TF-IDF alter the results?

TF-IDF assigned more weight to words that are highly prevalent in a single document but comparatively rare across the corpus. As a result, comparatively more “significant” words in each text are weighted more heavily by TF-IDF (and perhaps some ubiquitous positive words like “gain” or “free” were weighted lightly because of their ubiquity).

3. Which specific words drove the changes?

While there were a few high TF-IDF positive words, like “passion,” “divine,” “affection,” “humane,” and “orderly,” almost every other high TF-IDF word was negative, like “imposition,” “impertinent,” “disorderly,” “imperfect,” “meddle,” “kill,” “scandalous,” “wrong,” “forbid,” “strictly,” “petty,” “idle,” “deficient,” et cetera.

4. When would each method be more appropriate? What limitation does document length create for the raw-count sentiment measure?

Raw counts might be more appropriate if the exact absolute frequency of certain words mattered more, especially with very short documents. On the other hand, for longer documents, raw counts are made less effective because of the inherent redundancy as texts get longer (because there are only so many function words in the English language); in such situations, as well as when there are mismatches between document lengths (as longer documents will have higher raw counts in general, and thus higher numbers of sentiment words), TF-IDF is more appropriate. Another situation where TF-IDF is preferable is when there are high-frequency words across all texts that thus reveal very little about their relative importance to specific documents (like if every document has the same positive language preamble, or header, or a similar convention, for example).

Section V (Challenge)

```
# Summing positive and negative TF-IDF sums from previous code
total_tfidf_sentiment <- words_tfidf_and_count |>
  mutate(total_tf_idf_sentiment = positive_tf_idf_sum + negative_tf_idf_sum) |>
  select(doc_title, total_tf_idf_sentiment)
```

```
total_tfidf_sentiment
```

```
# A tibble: 2 x 2
  doc_title total_tf_idf_sentiment
  <chr>          <dbl>
1 Text A          149.
2 Text B           75.5
```

```
# Determining total sentiment of top 5 sentiment words in each text
top5_tfidf_sentiment <- words_tfidf_sentiment |>
  mutate(overall_rank = min_rank(desc(tf_idf))) |>
  group_by(doc_id) |>
  slice_max(tf_idf, n = 5, with_ties = FALSE) |>
  mutate(total_top5_sentiment = sum(tf_idf)) |>
  mutate(doc_title = case_when(doc_id == "text1" ~ "Text A", doc_id == "text2" ~ "Text B" ))
  ungroup() |>
  select(doc_title, total_top5_sentiment) |>
  distinct()

top5_tfidf_sentiment
```

```
# A tibble: 2 x 2
  doc_title total_top5_sentiment
  <chr>          <dbl>
1 Text A          15.7
2 Text B          10.3
```

```
# Joining top 5 and total sentiment, calculating proportion
inner_join(top5_tfidf_sentiment, total_tfidf_sentiment, by = join_by(doc_title)) |>
  mutate(prop_top5_sentiment = total_top5_sentiment / total_tf_idf_sentiment)
```

```
# A tibble: 2 x 4
  doc_title total_top5_sentiment total_tf_idf_sentiment prop_top5_sentiment
  <chr>          <dbl>          <dbl>          <dbl>
1 Text A          15.7          149.           0.105
2 Text B          10.3           75.5           0.136
```

In Text A, 10.48% of the sentiment is carried by the top 5 words. In Text B, 13.59% of the sentiment is carried in the top 5 words.